



Desarrollo del Portal e Interfaz para el Proyecto TeresIA

E2-Ingesta: Despliegue de OntoPortal con Ontologías y Validación de Capacidades

4 de septiembre de 2025



Resumen Ejecutivo	4
Objetivos y Alcance del Hito	5
Proceso de Despliegue y Carga	5
Preparación del Entorno	5
Carga de las Ontologías	6
Indexación y Anotación	8
Validación desde la Interfaz Web	9
Validación de la carga	9
Exploración de Clases y Propiedades	10
Búsqueda desde la Interfaz	11
Anotación desde la Interfaz	12
Visualización de Grafos	13
Validación de Procesos	13
Ingestión de ficheros RDF/Turtle	13
Indexación en Solr	14
Verificación de Endpoints de la API REST	15
Interfaz Web	16
Documentación del Proceso de Carga	16
Evaluación de Capacidades Funcionales	19
Identificación de Limitaciones	21
Falta inicial del procesador de tareas (NCBO Cron)	21
Visualización de propiedades en la interfaz	22
Búsqueda desde el frontend	22
Visualización de grafos	22
Dependencia de URLs codificadas	23
Capacidades Identificadas	23
Funcionalidades Válidas	23
Funcionalidades Por Corregir	24
Funcionalidades Para Implementar	24
Plan para el Próximo Hito	25
Conclusiones	25
Anexo: Solución del uso de URLs estáticas de BioPortal	26
Ontologies_linked_Data	26
Ontologies_api_ruby_client	28



Resumen Ejecutivo

Este segundo entregable tiene como propósito validar la infraestructura desplegada en el primer hito mediante la carga y procesamiento de las primeras ontologías de prueba en el portal TeresIA. Para ello se han utilizado los ficheros *aparells_sanitarios_rdf.ttl* y *dispositivos_moviles_rdf.ttl*, lo que ha permitido comprobar el flujo completo de ingestión, indexación y consulta de terminologías dentro de OntoPortal.

El objetivo principal de esta fase ha sido verificar la correcta gestión de ontologías en formato RDF/Turtle, tanto a través de la API como desde la interfaz web. Se han documentado los pasos de configuración del entorno, registro de ontologías y validación de los endpoints de búsqueda y anotación.

Los resultados confirman que OntoPortal admite de forma estable la carga y visualización de terminologías, con consultas básicas disponibles tanto desde el catálogo como desde la API. Durante el proceso se identificaron algunas incidencias relacionadas con la visualización de propiedades desde el frontend y la gestión de URLs codificadas rígidamente.

En conjunto, el entregable proporciona una primera versión operativa del catálogo de terminologías y un inventario de las funcionalidades verificadas, los aspectos pendientes de corrección y las capacidades planificadas para desarrollos futuros.



Objetivos y Alcance del Hito

El segundo hito se ha centrado en validar de manera práctica la infraestructura desplegada en la fase inicial del proyecto. Mientras que el primer entregable se orientó a garantizar la instalación y configuración de OntoPortal como núcleo tecnológico de la plataforma, en esta fase el objetivo ha sido comprobar el ciclo completo de ingestión y procesamiento de ontologías en formato RDF/Turtle.

De acuerdo con el alcance definido en la planificación inicial, los objetivos específicos de este hito han sido:

- Verificar la correcta **carga de ontologías de prueba** a través de la interfaz web del portal.
- Asegurar el **procesamiento automático de las ontologías** mediante la integración del servicio NCBO Cron, encargado de ejecutar las tareas en segundo plano.
- Confirmar la **indexación de clases y propiedades en Solr**, así como la disponibilidad de las funcionalidades de búsqueda y anotación desde la API y la interfaz.
- Documentar de manera detallada los pasos de configuración y despliegue, de forma que puedan ser reproducidos en fases posteriores.
- Identificar las **limitaciones técnicas** detectadas durante el proceso, con el fin de planificar las correcciones necesarias en los siguientes hitos.

Como parte de este proceso se incluyó la carga inicial de dos ontologías de prueba (*aparells_sanitarios_rdf.ttl* y *dispositivos_moviles_rdf.ttl*). Esta acción permitió comprobar la interacción entre los distintos componentes del sistema y sirvió para validar el correcto funcionamiento de la cadena de ingestión, desde la subida del fichero hasta la disponibilidad de los recursos en el catálogo y en los endpoints de la API.

La incorporación del servicio NCBO Cron resultó esencial para completar el procesamiento, ya que la configuración inicial de OntoPortal no contemplaba este componente. Una vez integrado, las ontologías pasaron por todas las fases previstas (*UPLOADED*, *RDF*, *INDEXED*, *ANNOTATOR*, entre otras), lo que permitió disponer de un primer inventario funcional sobre el que avanzar en el desarrollo del portal.

Proceso de Despliegue y Carga

Esta sección describe de forma detallada los pasos realizados para habilitar el funcionamiento completo de OntoPortal en el marco del proyecto TeresIA. El trabajo se estructuró en cuatro fases: preparación del entorno, carga de ontologías de prueba, verificación de la indexación y anotación, y validación desde la interfaz de usuario.

Preparación del Entorno

El despliegue se realizó en un entorno local, ya que en esta fase aún no estaba disponible el entorno remoto. Para ello se siguieron las instrucciones del repositorio principal y se ejecutaron los siguientes pasos:

1. Creación del fichero de configuración `.env` a partir de la plantilla `env.sample`.

```
$ cp .env.sample .env
```



2. Despliegue de los servicios principales mediante Docker Compose, utilizando el perfil basado en AllegroGraph.

```
$ docker compose --profile agraph up -d
```

1. Creación de un usuario administrador mediante el script `create_admin_user.rb`, obteniendo la correspondiente API Key.

```
$ docker compose exec api-agraph bash -c "ruby create_admin_user.rb"
```

2. Incorporación de la API Key en el fichero `.env` y reinicio del servicio del frontend para validar la conexión con la API.

```
$ docker compose restart frontend-dev
```

Durante esta etapa se identificó que la configuración inicial no contemplaba el servicio NCBO Cron, necesario para procesar en segundo plano las tareas de indexación y anotación de ontologías. Para resolverlo se añadieron dos nuevos servicios al archivo `docker-compose.yml` (`ncbo-cron-worker` y `ncbo-cron-worker-4store`), configurados para interactuar con la API en los perfiles de AllegroGraph y 4Store.

Además, se integró la gema de Ruby `sys-proctable`, requerida por NCBO Cron, y se modificaron los ficheros de configuración `development.rb` y `production.rb` para añadir las variables necesarias. Finalmente, se creó el script `start_ncbo_cron.sh` para inicializar el worker en el contenedor y se definieron dos variables adicionales en el fichero `.env` del frontend, diferenciando las llamadas a la API que se realizan desde el cliente y desde el backend

Carga de las Ontologías

Una vez configurado el entorno local y habilitado el servicio NCBO Cron, se procedió a la incorporación de las primeras ontologías de prueba: `aparells_sanitarios_rdf.ttl` y `dispositivos_moviles_rdf.ttl`. La carga se realizó a través de la interfaz web de OntoPortal, utilizando el formulario "Submit Ontology". En este formulario se especificaron los metadatos principales (nombre, acrónimo, visibilidad y descripción), junto con el archivo de la ontología y el formato de representación. Se optó por OWL, dado que la OWL-API empleada por OntoPortal admite los formatos RDF/Turtle.



Submit new ontology

Details General Date and contacts

Name*
aparells-sanitarios

Acronym*
AS

Visibility*
public

Administrators*
admin

Categories

Groups

Is this ontology a view of another ontology? ☐

Next

Description

Representation language*
OWL

Status
production

Location*
☐ Metadata Only
☐ Allow Users
☐ Load From URI
☐ New Versions Loaded
☒ Upload Local File

aparells-sanitarios.owl

Figura 1: Formulario de carga de ontologías en la interfaz de OntoPortal.

Tras la subida inicial, las ontologías quedaban registradas en el catálogo, pero su procesamiento no se completaba hasta que el servicio NCBO Cron ejecutaba la cola de tareas en Redis. Una vez finalizado este procesamiento, fue posible acceder a los datos enriquecidos: clases, propiedades, métricas, notas y mappings, además de la opción de descarga en diferentes formatos.

aparells-sanitarios2
Last uploaded: August 27, 2025

Summary

Details

Acronym	AS2
Visibility	Public
Description	
Status	Production
Format	OWL
Contact	Markus (markus.schuller@laiw.es)
Creation date	August 27, 2025
Deprecated	False
Object creation date property	http://purl.org/dc/elements/1.1/created
Object modification date property	http://purl.org/dc/elements/1.1/modified
Submission date	August 27, 2025
URI Lookup Endpoint	http://api-agraph:9393/search?ontologies=AS2&require_exact_match=true&q=

Metrics

We have not yet calculated metrics for AS2

Visits

We are still collecting data for AS2

Projects using AS2

No projects using AS2

Submissions

Version	Released	Uploaded	Downloads
unknown (Uploaded)	08/27/2025	08/27/2025	OWL

Figura 2: Ejemplo de metadatos y métricas generados automáticamente tras el procesamiento de una ontología.

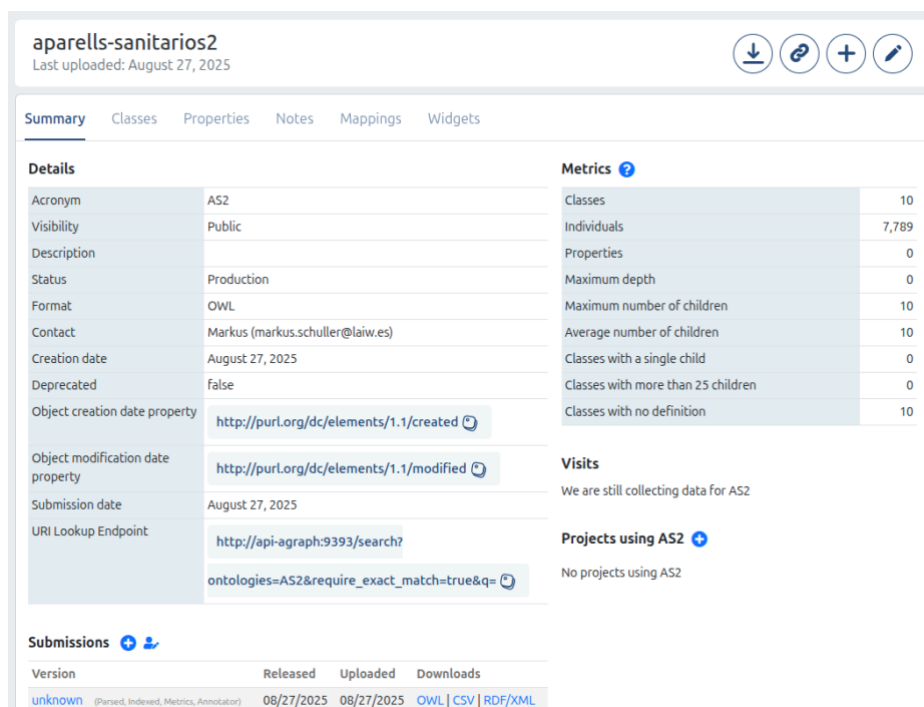


Figura 3: Ejemplo de metadatos y métricas generados automáticamente tras el procesamiento de una ontología.

La verificación del proceso se llevó a cabo mediante la API, utilizando el endpoint `GET /ontologies/{Acrónimo}/latest_submission?include=all`. En la respuesta se observó la secuencia de estados que confirman el éxito del procesamiento (`UPLOADED`, `RDF`, `RDF_LABELS`, `OBSOLETE`, `INDEXED`, `INDEXED_PROPERTIES`, `METRICS`, `ANNOTATOR`), lo que valida el correcto funcionamiento de la fase de ingesta.

También podemos acceder a los siguientes *endpoints* de la API para obtener los datos:

- **GET /ontologies**: Obtener un sumario de todas las ontologías creadas.
- **GET /ontologies/{Acrónimo}/latest_submission?include=all**: Obtener un detalle del último archivo subido de la ontología con el acrónimo especificado. Aquí podemos ver el estado de procesamiento de la ontología en la propiedad `submissionStatus`, en el que se añaden los tipos de procesamiento por los que ha pasado ("UPLOADED", "RDF", "RDF_LABELS", "OBSOLETE", "INDEXED", "INDEXED_PROPERTIES", "METRICS", "ANNOTATOR"...).
- **GET /ontologies/{Acrónimo}/[classes | properties | categories | notes | ...]**: Con diferentes variaciones de este endpoint podemos obtener ciertos metadatos de la ontología una vez se hayan procesado sus clases, propiedades, etc.

Indexación y Anotación

Tras el procesamiento de las ontologías, se verificó que los recursos se hubieran indexado y anotado correctamente en la infraestructura desplegada.



En primer lugar, se comprobó el funcionamiento del motor de búsqueda **Solr**, que en el entorno local cuenta con dos cores configurados: *term_search_core1* y *prop_search_core1*. Para ello se ejecutaron consultas de prueba:

- http://localhost:8983/solr/term_search_core1/select?q=*&fq=ontologyId:*AS*&fl=id,pr efLabel,ontologyId
Esta petición devolvió las diez clases correspondientes a la ontología *aparells_sanitarios*.
- http://localhost:8983/solr/prop_search_core1/select?q=*&fq=ontologyId:*AS*&fl=id,pr efLabel,ontologyId&rows=100
El resultado incluyó las veintinueve propiedades extraídas de la misma ontología.

También, podemos comprobar que el servicio de búsqueda funciona directamente desde la API, buscando por ejemplo la siguiente clase:

- <http://localhost:9393/search?q=Actor>
Esta petición nos devuelve la clase “Actor” que hay en la ontología AS.

Asimismo, se validó el servicio de anotación basado en **Mgrep** mediante el endpoint */annotator*. Con entradas de texto simples y compuestas se confirmó que el sistema identificaba de forma correcta términos de las ontologías, devolviendo las posiciones exactas de las coincidencias dentro del texto:

- <http://localhost:9393/annotator?text=concept>
Esta petición nos devuelve, por cada ontología, la clase anotada “Concept” con la anotación “CONCEPT” que abarca desde el carácter 1 al 7 (todo el texto introducido, ya que solo hemos escrito la palabra “concept”).
- <http://localhost:9393/annotator?text=The%20definition%20of%20a%20concept>
Esta petición es un poco más compleja, ya que nos devuelve la misma clase “Concept” con la misma anotación, pero en este caso identifica que abarca desde el carácter 21 al 27.

Validación desde la Interfaz Web

Validación de la carga

La incorporación de ontologías se realizó íntegramente desde la interfaz de OntoPortal mediante el formulario “Submit Ontology”. En cada carga se completaron los metadatos básicos (nombre, acrónimo, visibilidad y, cuando procedía, descripción), se adjuntó el archivo de la ontología y se seleccionó OWL como formato de representación, aprovechando la compatibilidad de OWL-API con RDF/Turtle.

Tras confirmar el formulario, la ontología quedó visible en el catálogo y se generó su página de detalle con el resumen de metadatos aportados. Una vez que NCBO Cron completó el procesamiento en segundo plano, se habilitaron las secciones derivadas del procesado: clases, propiedades, métricas, notas, mappings y widgets, además de las opciones de descarga en varios formatos. Desde esta misma vista fue posible editar metadatos (icono de lápiz) y cargar nuevas versiones (icono “+”).

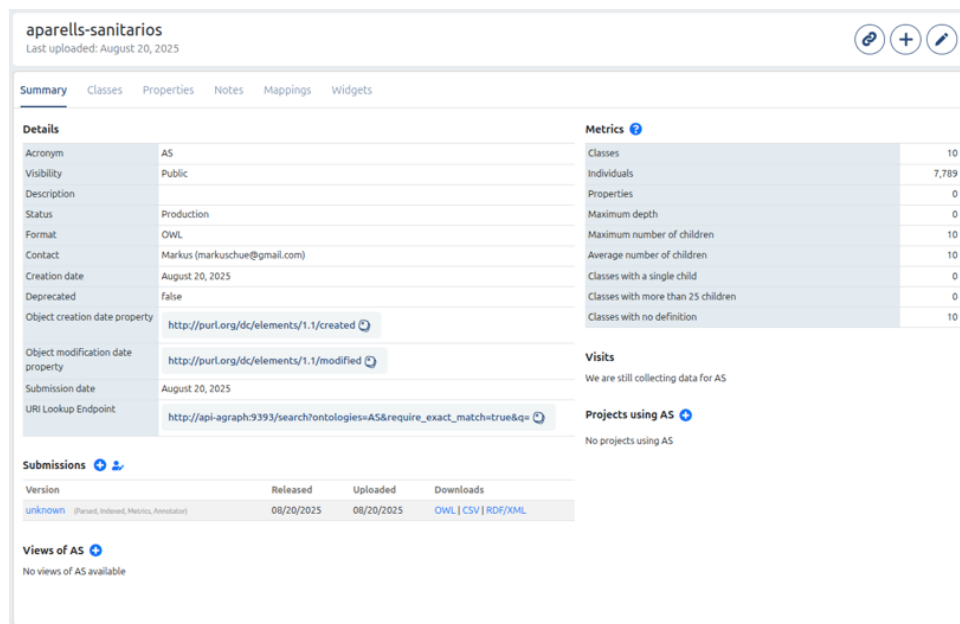


Figura 4: Página de detalle de una ontología recién cargada antes/después del procesamiento (metadatos y secciones habilitadas).

Exploración de Clases y Propiedades

La pestaña “Classes” permitió navegar por la jerarquía de clases y consultar los metadatos resultantes del procesamiento. En la pestaña “Properties”, pese a que los datos estaban disponibles en la API, inicialmente no se mostraban en la interfaz debido al uso de URLs codificadas hacia BioPortal en componentes del frontend. La corrección consistió en sustituir dichas URLs por variables de entorno configurables, con lo que la interfaz pasó a resolver correctamente la ruta de la API del despliegue local y a representar la lista de propiedades sin incidencias.

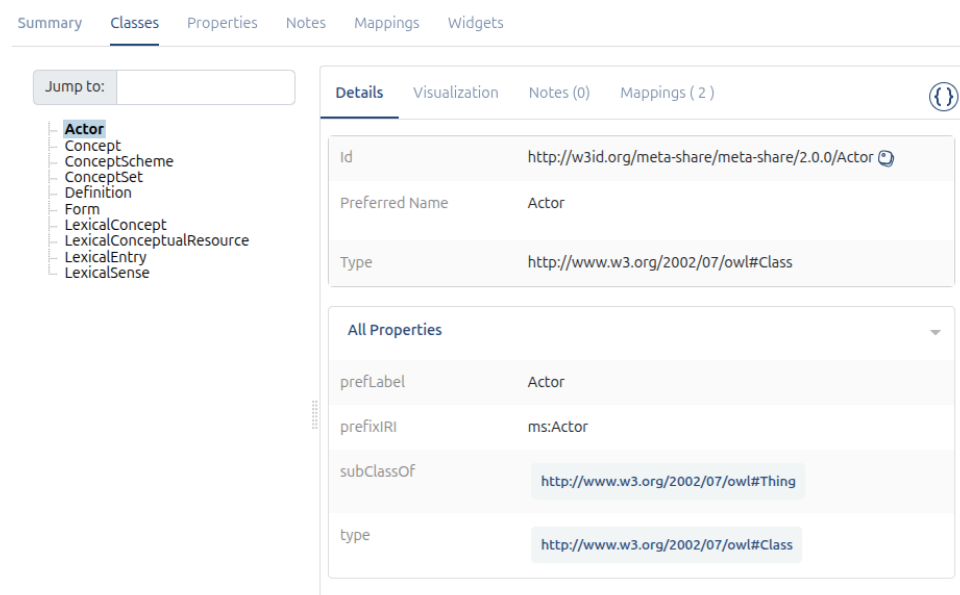


Figura 5: Pestaña “Classes” mostrando la jerarquía y metadatos de las clases de la ontología.

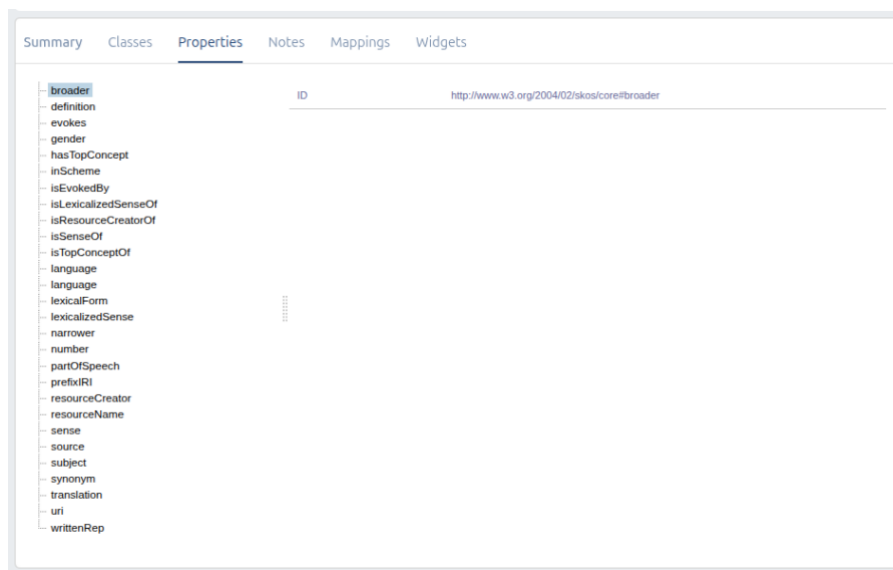


Figura 6: Pestaña “Properties” tras la corrección de URLs, con las propiedades ya visibles.

Búsqueda desde la Interfaz

Se validó la búsqueda de clases y términos desde la barra de búsqueda del frontend. En las primeras pruebas, la resolución de la API fallaba en el navegador porque algunas peticiones cliente apuntaban a la dirección interna de Docker. Se adoptó una configuración diferenciada de URLs de API: una interna para llamadas servidor-a-servidor y otra pública para las llamadas iniciadas en el cliente (navegador). Este ajuste permitió completar las consultas desde la interfaz y alinear los resultados con los devueltos por la API. Esta solución implica exponer la API mediante una URL pública; se consideró aceptable en este hito dado que la API ya exige API Keys y controles asociados.

Class Search

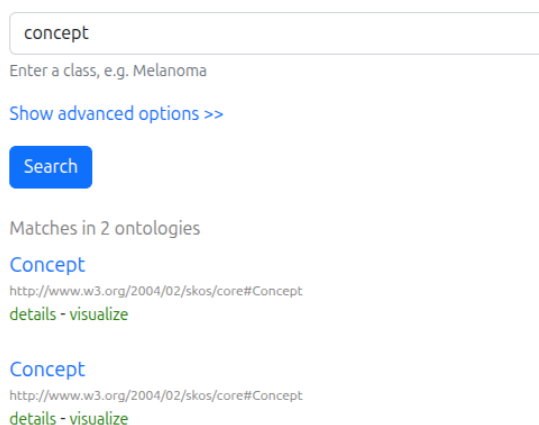


Figura 7: Resultados de una búsqueda de clases desde la interfaz (consulta y listado de coincidencias).



Anotación desde la Interfaz

El servicio de anotación se probó en la interfaz con textos simples y en frases con contexto. Al inicio se reprodujeron errores análogos a los de “Properties” por la presencia de URLs codificadas de BioPortal en el frontend y en la librería cliente utilizada.

Annotations

total results 2 (direct 2 / ancestor 0 / mapping 0)

CLASS	filter	ONTOLOGY	filter	TYPE	filter	CONTEXT	MATCHED CLASS	MATCHED ONTOLOGY
Concept		aparells-sanitarios		direct		... of a concept	Concept	aparells-sanitarios
Concept		dispositivos-moviles		direct		... of a concept	Concept	dispositivos-moviles

Figura 8: Resultado de anotación en la interfaz para un texto de ejemplo, con términos detectados y posiciones.

Tras generalizar la configuración mediante variables de entorno y asegurando que todas las llamadas apuntasen a la API del despliegue, la interfaz mostró correctamente las anotaciones devueltas por el servicio (término identificado y rangos de caracteres). Se documentaron ejemplos con textos como “concept” y “The definition of a concept”, verificando la coherencia entre interfaz y respuesta de la API.

Visualización de Grafos

Los componentes de visualización de grafos dependen de BioMixer, un servicio externo de la organización de OntoPortal. En el entorno local, la visualización no puede completarse porque BioMixer espera que la ontología sea accesible a través de la API pública de BioPortal. Como medida provisional se configuró la URL de BioPortal para evitar errores de renderizado en la página, aunque los grafos no se muestran correctamente en este contexto. Dado que BioMixer está orientado a ontologías biomédicas y no es viable su uso directo en este despliegue, se plantea para fases posteriores evaluar un fork adaptado a la API del proyecto o integrar una solución alternativa de visualización de grafos más alineada con los requisitos de Teresia.

Validación de Procesos

En este hito se validó el ciclo completo de gestión de las ontologías en OntoPortal, desde su ingesta hasta la consulta final a través de la interfaz y la API. El proceso incluyó varias comprobaciones específicas:

Ingestión de ficheros RDF/Turtle

La carga de los ficheros de prueba (*aparells_sanitarios_rdf.ttl* y *dispositivos_moviles_rdf.ttl*) se realizó sin incidencias una vez que el servicio **NCBO Cron** estuvo operativo. Este componente permitió procesar en segundo plano las tareas asociadas a la conversión de las ontologías y a la generación de sus metadatos.



La verificación del flujo se efectuó a través del endpoint:

GET /ontologies/{Acrónimo}/latest_submission?include=all

```
{
  id: http://api-agraph:9393/ontologies/AS/submissions/1,
  creationDate: "2025-08-27T00:00:00.000+00:00",
  status: "production",
  submissionId: 1,
  hasOntologyLanguage: "OWL",
+ ontology: { ... },
- submissionStatus: [
  "UPLOADED",
  "RDF",
  "RDF_LABELS",
  "OBSOLETE",
  "INDEXED",
  "INDEXED_PROPERTIES",
  "METRICS",
  "ANNOTATOR"
],
}
```

Figura 9: Captura de la respuesta JSON recibida a la consulta GET sobre la última ontología indexada.

En la respuesta se observaron los distintos estados del procesamiento: *UPLOADED*, *RDF*, *INDEXED* y *ANNOTATOR*, lo que confirma que las etapas de la cadena de ingestión se ejecutaron de manera completa y en el orden esperado.

Indexación en Solr

La indexación de los recursos se comprobó mediante consultas directas a los cores de **Solr**:

- **Clases:** http://localhost:8983/solr/term_search_core1/select?q=*&fq=ontologyId:*AS*
Esta consulta devolvió las diez clases registradas en la ontología *aparells_sanitaris*, mostrando los campos id, prefLabel y ontologyId.



```
{
  "responseHeader": {
    "status": 0,
    "QTime": 1,
    "params": {
      "q": "*",
      "fl": "id,prefLabel,ontologyId",
      "fq": "ontologyId:*AS*"}},
  "response": {
    "numFound": 10, "start": 0, "numFoundExact": true, "docs": [
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/ns/lemon/ontolex#LexicalSense_AS_1",
        "prefLabel": ["LexicalSense"]},
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://w3id.org/meta-share/meta-share/2.0.0/Actor_AS_1",
        "prefLabel": ["Actor"]},
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/ns/lemon/ontolex#LexicalConcept_AS_1",
        "prefLabel": ["LexicalConcept"]},
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/ns/lemon/ontolex#ConceptSet_AS_1",
        "prefLabel": ["ConceptSet"]},
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "https://termlex.oeg.fi.upm.es/termlex/Definition_AS_1",
        "prefLabel": ["Definition"]},
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/2004/02/skos/core#Concept_AS_1",
        "prefLabel": ["Concept"]},
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/2004/02/skos/core#ConceptScheme_AS_1",
        "prefLabel": ["ConceptScheme"]},
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://w3id.org/meta-share/meta-share/2.0.0/LexicalConceptualResource_AS_1",
        "prefLabel": ["LexicalConceptualResource"]},
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/ns/lemon/ontolex#Form_AS_1",
        "prefLabel": ["Form"]},
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/ns/lemon/ontolex#LexicalEntry_AS_1",
        "prefLabel": ["LexicalEntry"]}]}
}
```

Figura 10: Resultado de la consulta a `term_search_core1` con las clases de la ontología `aparells_sanitaris`.

- **Propiedades:** http://localhost:8983/solr/prop_search_core1/select?q=*&fq=ontologyId:*AS*&rows=100

El resultado incluyó las veintinueve propiedades extraídas, confirmando la correcta indexación en el core correspondiente.



```
{
  "responseHeader": {
    "status": 0,
    "QTime": 1,
    "params": {
      "q": "*",
      "fl": "id,prefLabel,ontologyId",
      "fq": "ontologyId:AS*",
      "rows": "100"
    }
  },
  "response": {
    "numFound": 29,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/2004/02/skos/core#broader_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/2004/02/skos/core#definition_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/ns/lemon/ontolex#evokes_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.lexinfo.net/ontology/3.0/lexinfo#gender_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/2004/02/skos/core#hasTopConcept_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/2004/02/skos/core#inScheme_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/ns/lemon/ontolex#isEvokedBy_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/ns/lemon/ontolex#isLexicalizedSenseOf_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://w3id.org/meta-share/meta-share/2.0.0/isResourceCreatorOf_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/ns/lemon/ontolex#isSenseOf_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://www.w3.org/2004/02/skos/core#isTopConceptOf_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://purl.org/dc/terms/language_AS_1"
      },
      {
        "ontologyId": "http://api-agraph:9393/ontologies/AS/submissions/1",
        "id": "http://w3id.org/meta-share/meta-share/2.0.0/language_AS_1"
      }
    ]
  }
}
```

Figura 11: Resultado de la consulta a `prop_search_core1` con las propiedades indexadas de la ontología `aparells_sanitaris`.

Verificación de Endpoints de la API REST

Se comprobó que los endpoints principales de la API respondieran conforme a lo esperado:

- GET `/ontologies`

Devolvió el listado de ontologías registradas.

- GET `/ontologies/{acrónimo}`

Permitió recuperar la información detallada de una ontología específica.

- GET `/search?q=Actor`

Recuperó correctamente la clase “Actor” en la ontología AS.

- GET `/annotator?text=concept`

Identificó la clase “Concept” en el texto de prueba, mostrando el rango de caracteres correspondiente.



Interfaz Web

La interfaz permitió validar las operaciones básicas de navegación y consulta. Fue posible:

- Acceder al listado de ontologías en el catálogo.
- Revisar los metadatos y métricas generados tras el procesamiento.
- Explorar la jerarquía de clases y visualizar las propiedades asociadas a cada ontología, una vez corregido el problema de URLs codificadas hacia BioPortal.
- Acceder a la sección de widgets para generar elementos de integración (autocompletado, visualización de clases o jerarquías).

La verificación desde la interfaz confirmó la coherencia entre la información mostrada en pantalla y los resultados proporcionados por la API.

Documentación del Proceso de Carga

En este apartado se recoge de forma detallada el procedimiento técnico seguido para desplegar y configurar los servicios de OntoPortal y ejecutar la carga inicial de ontologías en el entorno local. El objetivo es garantizar la reproducibilidad del proceso en futuras fases del proyecto.

- **Comandos ejecutados:** Durante la preparación y puesta en marcha del entorno se emplearon los siguientes comandos:

1. `cp .env.sample .env` - Para crear el fichero de entorno.
2. `docker compose --profile agraph up -d` - Para levantar los servicios principales con el perfil de AllegroGraph.
3. `docker compose exec api-agraph bash -c "ruby create_admin_user.rb"` - Para crear el usuario administrador y obtener su API Key.
4. `docker compose restart frontend-dev` - Para reiniciar el frontend tras añadir la API Key.

- **Ficheros añadidos o modificados:** Se realizaron los siguientes cambios en la configuración del sistema:

5. Inclusión de la API Key del administrador en el fichero `.env`:

```
# Obtenida de ejecutar el script create_admin_user.rb
# Cada vez que se reinicie la base de datos, este valor debe ser
actualizado
API_KEY=c913840e-f31d-4357-ab62-92443ffea319
```

1. Adición de variables de configuración específicas para el servicio NCBO Cron en `config/environments/development.rb` y `config/environments/production.rb`:

```
NcboCron.config do |config|
  config.redis_host = ENV.fetch("REDIS_HOST", "redis-ut")
```



```
config.redis_port = REDIS_PORT.to_i
config.ontology_report_path = REPORT_PATH.to_s
# Do not daemonize in Docker
config.daemonize = false
# Processing intervals - check every minute
config.minutes_between = 1
config.enable_ontology_analytics = false
config.search_index_all_url =
'http://localhost:8983/solr/term_search_core2'
config.property_search_server_index_all_url =
'http://localhost:8983/solr/prop_search_core2'
config.ontology_report_path =
"#{${DATADIR}}/reports/ontologies_report.json"
config.enable_spam_deletion = false
end
```

6. Inclusión de la gema de Ruby **sys-proctable** en el fichero Gemfile, necesaria para la ejecución del servicio NCBO Cron:

```
gem 'sys-proctable'
```

7. Definición de dos nuevos servicios en docker-compose.yml para habilitar el procesamiento en segundo plano:

```
ncbo-cron-worker:
  <<: *api-app
  command: bash /srv/ontoportal/ontologies_api/start-ncbo-cron.sh
  environment:
    <<: *api-env
    REST_URL_PREFIX: ${REST_URL_PREFIX:-http://api-
agraph:9393}
    GOO_BACKEND_NAME: ag
    GOO_PORT: 10035
    GOO_HOST: agraph-ut
    GOO_PATH_QUERY: /repositories/ontoportal_test
    GOO_PATH_DATA: /repositories/ontoportal_test/statements
    GOO_PATH_UPDATE: /repositories/ontoportal_test/statements
  profiles:
    - agraph
  depends_on:
    <<: *api-depends-on
    agraph-ut:
      condition: service_healthy
    api-agraph:
      condition: service_healthy
  networks:
    - internal
  volumes:
    - api-bundle:/srv/ontoportal/bundle
    - ./api:/srv/ontoportal/ontologies_api
```




8. Creación del script `start_ncbo_cron.sh`, encargado de configurar el entorno de ejecución e iniciar el proceso `ncbo_cron`:

```
#!/usr/bin/env bash
cd /srv/ontoportal/ontologies_api
# Configure bundle
bundle config set --local path /srv/ontoportal/bundle
# Install gems quietly
bundle install --quiet
# Find the ncbo_cron executable path
NCBO_CRON_PATH=$(find /srv/ontoportal/bundle -name ncbo_cron -
type f | head -1)
echo "Found ncbo_cron at: $NCBO_CRON_PATH"
echo "Starting ncbo_cron with minimal API initialization..."
# Create a minimal wrapper that only loads what ncbo_cron needs
cat > /tmp/ncbo_cron_minimal_wrapper.rb << 'EOF'
#!/usr/bin/env ruby
# Load bundler setup
require 'bundler/setup'
# Load only the essential libraries that define
LinkedData::OntologiesAPI
require 'ontologies_linked_data'
# Create a minimal LinkedData module structure if not already
defined
unless defined?(LinkedData::OntologiesAPI)
  module LinkedData
    module OntologiesAPI
      def self.config
        yield self if block_given?
      end
      def self.method_missing(method, *args, &block)
        # Stub for configuration methods
      end
    end
  end
end
# Now load and run ncbo_cron
load ARGV[0]
EOF
# Execute the minimal wrapper with the ncbo_cron path
bundle exec ruby /tmp/ncbo_cron_minimal_wrapper.rb
"$NCBO_CRON_PATH" --log-level info & tail -f
/srv/ontoportal/ontologies_api/log/scheduler.log
```

9. Incorporación de dos variables de URL en el fichero `.env` del frontend, diferenciando el uso interno y externo de la API:

```
# API Configuration
API_URL=http://api-agraph:9393/
EXTERNAL_API_URL=http://localhost:9393
```



Estos cambios garantizaron la correcta ejecución del proceso de ingestión y la disponibilidad de los servicios de indexación y anotación dentro del entorno de desarrollo local.

Evaluación de Capacidades Funcionales

Tras completar el proceso de despliegue y la carga de las primeras ontologías de prueba, se procedió a la verificación de las principales funcionalidades disponibles en OntoPortal. El objetivo de esta fase fue determinar qué componentes del sistema se encontraban plenamente operativos y cuáles requerían ajustes o desarrollos adicionales.

Las capacidades validadas se resumen a continuación:

Registro y procesamiento de ontologías: Se comprobó que las ontologías en formato RDF/Turtle se registran correctamente en el portal a través de la interfaz web. Una vez cargadas, el servicio NCBO Cron ejecutó el procesamiento en segundo plano, generando las estructuras necesarias (clases, propiedades, métricas y metadatos) y actualizando el estado de la *submission* en la API. Esto permitió confirmar que el flujo de ingestión está correctamente habilitado.

Catálogo de ontologías: Las ontologías cargadas, como *aparells_sanitaris* y *dispositivos_moviles*, aparecen listadas en la sección “Ontologies” del portal. Su información es accesible tanto desde la interfaz de usuario como a través del endpoint /ontologies de la API, lo que confirma la correcta sincronización entre el catálogo y el backend.

0 dispositivos-moviles (DS)		instances	classes
null		2,164	10
Uploaded: 8/27/25			

1 aparells-sanitaris (AS)		instances	classes
null		7,789	10
Uploaded: 8/27/25			

Figura 12: Catálogo de las ontologías cargadas que se muestran en la sección “Ontologies”.

Búsqueda básica: Se validó el funcionamiento de la búsqueda de términos y clases. Consultas de ejemplo, como “Actor”, devolvieron los resultados esperados en el frontend y a través del endpoint /search de la API. Esta prueba demostró que los mecanismos de indexación en Solr y la comunicación entre el cliente y la API funcionan de manera adecuada tras la corrección de las URLs internas de Docker.



Class Search

Enter a class, e.g. Melanoma

[help](#)

[Show advanced options >>](#)

Matches in 2 ontologies

[Concept - aparells-sanitaris \(AS\)](#)

<http://www.w3.org/2004/02/skos/core#Concept>

[details](#) - [visualize](#)

[Concept - dispositius-movils \(DS\)](#)

<http://www.w3.org/2004/02/skos/core#Concept>

[details](#) - [visualize](#)

Figura 13: Ejemplo de búsqueda desde la interfaz web.

Recuperación de información vía API REST: Los endpoints principales de OntoPortal fueron comprobados con éxito:

- /ontologies para la obtención del listado de ontologías.
- /ontologies/{acrónimo} para acceder a los detalles de cada ontología.
- /search para la recuperación de términos y clases.
- /annotator para la anotación de fragmentos de texto.

En todos los casos, la API devolvió la información en el formato esperado, incluyendo el estado de procesamiento de las ontologías y los resultados de búsqueda y anotación.



```
{
  "administeredBy": {
    "http://api-agraph.9393/users/admin"
  },
  "acronym": "DS",
  "name": "dispositivos-moviles",
  "summaryOnly": false,
  "flat": null,
  "ontologyType": "http://api-agraph.9393/ontology_types/ONTOLOGY",
  "id": "http://api-agraph.9393/ontologies/DS",
  "gtype": "http://api-agraph.9393/metadata/Ontology",
  "links": {
    "submissions": "http://api-agraph.9393/ontologies/DS/submissions",
    "properties": "http://api-agraph.9393/ontologies/DS/properties",
    "classes": "http://api-agraph.9393/ontologies/DS/classes",
    "single class": "http://api-agraph.9393/ontologies/DS/classes/class_id",
    "roots": "http://api-agraph.9393/ontologies/DS/classes/roots",
    "schemes": "http://api-agraph.9393/ontologies/DS/schemes",
    "collections": "http://api-agraph.9393/ontologies/DS/collections",
    "xl labels": "http://api-agraph.9393/ontologies/DS/skos_xl_labels",
    "instances": "http://api-agraph.9393/ontologies/DS/instances",
    "metrics": "http://api-agraph.9393/ontologies/DS/metrics",
    "reviews": "http://api-agraph.9393/ontologies/DS/reviews",
    "notes": "http://api-agraph.9393/ontologies/DS/notes",
    "groups": "http://api-agraph.9393/ontologies/DS/groups",
    "categories": "http://api-agraph.9393/ontologies/DS/categories",
    "latest submission": "http://api-agraph.9393/ontologies/DS/latest_submission",
    "projects": "http://api-agraph.9393/ontologies/DS/projects",
    "download": "http://api-agraph.9393/ontologies/DS/download",
    "views": "http://api-agraph.9393/ontologies/DS/views",
    "analytics": "http://api-agraph.9393/ontologies/DS/analytics",
    "ui": "http://localhost:3008/ontologies/DS",
    "@context": {
      "submissions": "http://api-agraph.9393/metadata/OntologySubmission",
      "properties": "http://api-agraph.9393/metadata/Property",
      "classes": "http://www.w3.org/2002/07/owl#Class",
      "single class": "http://www.w3.org/2002/07/owl#Class",
      "roots": "http://www.w3.org/2002/07/owl#Class",
      "schemes": "http://www.w3.org/2004/02/skos/core#ConceptScheme",
      "collections": "http://www.w3.org/2004/02/skos/core#Collection",
      "xl labels": "http://www.w3.org/2008/05/skos-xl#label",
      "instances": "http://api-agraph.9393/metadata/Instance",
      "metrics": "http://api-agraph.9393/metadata/Metrics",
      "reviews": "http://api-agraph.9393/metadata/Review",
      "notes": "http://api-agraph.9393/metadata/Note",
      "groups": "http://api-agraph.9393/metadata/Group",
      "categories": "http://api-agraph.9393/metadata/Category",
      "latest submission": "http://api-agraph.9393/metadata/OntologySubmission",
      "projects": "http://api-agraph.9393/metadata/Project",
      "download": "http://api-agraph.9393/metadata/Ontology",
      "views": "http://api-agraph.9393/metadata/Ontology",
      "analytics": "http://api-agraph.9393/metadata/Analytics",
      "ui": "http://api-agraph.9393/metadata/Ontology"
    }
  },
  "page": 1,
  "pageCount": 1,
  "totalCount": 2,
  "prevPage": null,
  "nextPage": null,
  "links": {
    "nextPage": null,
    "prevPage": null
  },
  "collection": [
    {
      "prefLabel": "Concept",
      "obsolete": false,
      "matchType": "prefLabel",
      "ontologyType": "ONTOLOGY",
      "provisional": false,
      "id": "http://www.w3.org/2004/02/skos/core#Concept",
      "gtype": "http://www.w3.org/2002/07/owl#Class",
      "links": {
        "self": "http://api-agraph.9393/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept",
        "ontology": "http://api-agraph.9393/ontologies/AS",
        "children": "http://api-agraph.9393/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept",
        "parents": "http://api-agraph.9393/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept",
        "descendants": "http://api-agraph.9393/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept",
        "ancestors": "http://api-agraph.9393/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept",
        "instances": "http://api-agraph.9393/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept",
        "tree": "http://api-agraph.9393/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept",
        "notes": "http://api-agraph.9393/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept",
        "mappings": "http://api-agraph.9393/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept",
        "ui": "http://localhost:3008/ontologies/AS/classes/http://www.w3.org/2004/02/skos/core#Concept"
      },
      "@context": {
        "self": "http://www.w3.org/2002/07/owl#Class",
        "ontology": "http://api-agraph.9393/metadata/Ontology",
        "children": "http://www.w3.org/2002/07/owl#Class",
        "parents": "http://www.w3.org/2002/07/owl#Class",
        "descendants": "http://www.w3.org/2002/07/owl#Class",
        "ancestors": "http://www.w3.org/2002/07/owl#Class",
        "instances": "http://api-agraph.9393/metadata/Instance",
        "tree": "http://www.w3.org/2002/07/owl#Class",
        "notes": "http://api-agraph.9393/metadata/Note",
        "mappings": "http://api-agraph.9393/metadata/Mapping",
        "ui": "http://www.w3.org/2002/07/owl#Class"
      }
    },
    {
      "@context": {
        "vocab": "http://api-agraph.9393/metadata/",
        "prefLabel": "http://www.w3.org/2004/02/skos/core#prefLabel",
        "obsolete": "http://www.w3.org/2002/07/owl#obsolete",
        "@language": "en"
      },
      "prefLabel": "Concept",
      "obsolete": false,
      "matchType": "prefLabel"
    }
  ]
}
```

Figura 13: Ejemplo de respuesta en formato JSON desde API.

Navegación en la interfaz web: La interfaz de usuario mostró de manera coherente los resultados generados por la API. Fue posible acceder al resumen de cada ontología, explorar la jerarquía de clases y consultar las propiedades y widgets asociados. Tras la corrección de las referencias a URLs codificadas de BioPortal, la visualización de propiedades se realizó de forma adecuada.

Identificación de Limitaciones

Durante la ejecución de este hito se detectaron diversas incidencias y restricciones técnicas que conviene tener en cuenta para la evolución del sistema. A continuación, se detallan los principales aspectos identificados:

Falta inicial del procesador de tareas (NCBO Cron)

- **Problema:** la ausencia de este servicio impedía completar el procesamiento de las ontologías tras su carga inicial.
- **Causa:** En la configuración de despliegue incluida en el repositorio original de OntoPortal no se contemplaba el servicio NCBO Cron, encargado de procesar en segundo plano las tareas intensivas (carga, indexación y anotación de ontologías).
- **Solución:** Se añadieron los servicios *ncbo-cron-worker* y *ncbo-cron-worker-4store* en el archivo *docker-compose.yml*, junto con la configuración necesaria para su conexión a la API. Tras este ajuste, el procesamiento se completó correctamente.



Visualización de propiedades en la interfaz

- **Problema:** La pestaña “Properties” de la interfaz no mostraba resultados, a pesar de que las propiedades sí estaban disponibles en la API.
- **Causa:** Referencias a URLs de BioPortal codificadas de forma rígida en el código del frontend.
- **Solución:** Se sustituyeron dichas referencias por variables de entorno configurables, lo que permitió mostrar correctamente las propiedades de las ontologías cargadas.

Búsqueda desde el frontend

- **Problema:** Las consultas de búsqueda y anotación realizadas desde la interfaz no devolvían resultados.
- **Causa:** Algunas peticiones se dirigían a la dirección interna de Docker de la API, inaccesible desde el navegador.
- **Solución:** Se configuraron dos variables de entorno distintas para la URL de la API, diferenciando el acceso interno (backend) del acceso público (frontend). Aunque esta solución resolvió el problema, implica la necesidad de exponer la API mediante una URL pública. Este aspecto se considera una limitación técnica que deberá analizarse en fases posteriores, dado que la seguridad depende actualmente de la gestión de API Keys.

Visualización de grafos

- **Problema:** Los componentes de visualización de grafos no se mostraban en el entorno local.
- **Causa:** La interfaz utiliza BioMixer, un servicio externo de la infraestructura de BioPortal, que no puede acceder a las ontologías cargadas localmente.
- **Solución provisional:** Se configuró la URL pública de BioPortal para evitar errores en la carga de la página. Sin embargo, la visualización de grafos no se completó.
- **Consideraciones:** será necesario evaluar alternativas en próximas fases, ya sea mediante la adaptación de un fork de BioMixer que se conecte a la API local, o bien mediante la integración de otras herramientas de visualización de grafos más adecuadas para el contexto de TeresIA.

Dependencia de URLs codificadas

- **Problema:** Inconsistencias en despliegues locales o en entornos con dominios personalizados.
- **Causa:** Se identificó que varias partes del código (frontend, cliente Ruby de la API y gemas de modelado de datos) incluían referencias fijas a la URL de producción de BioPortal (<http://data.bioontology.org>).



- **Solución provisional:** Se implementó un parche para sustituir dichas URLs por las definidas en variables de entorno.
- **Mejora pendiente:** Se recomienda realizar un fork de las gemas afectadas y reemplazar las referencias estáticas por parámetros configurables, de modo que la solución sea más robusta y mantenible a largo plazo.

Capacidades Identificadas

Tras la ejecución de las pruebas de ingestión, indexación, anotación y validación en la interfaz, se ha obtenido un inventario inicial de las capacidades disponibles en el sistema, junto con los aspectos que requieren ajustes y las funcionalidades aún pendientes de desarrollo.

Funcionalidades Válidas

Se ha confirmado el correcto funcionamiento de los siguientes componentes y procesos:

- **Registro y procesamiento de ontologías:** Las ontologías en formato RDF/Turtle se registran correctamente mediante la interfaz web. Tras la carga, el servicio NCBO Cron ejecuta el procesamiento en segundo plano y actualiza el estado de la *submission*. Este flujo incluye la generación de clases, propiedades, métricas y metadatos asociados.
- **Catálogo de ontologías:** Las ontologías cargadas aparecen en la sección “Ontologies” del portal y su información se recupera de forma coherente tanto en la interfaz como a través del endpoint /ontologies de la API.
- **Indexación y búsqueda:** Los recursos de cada ontología (clases y propiedades) se indexan en los cores de Solr configurados. Las consultas básicas realizadas desde la API (/search) y desde la barra de búsqueda de la interfaz devuelven resultados correctos.
- **Servicio de anotación:** El módulo de anotación basado en Mgrep responde adecuadamente a las peticiones al endpoint /annotator, identificando de manera precisa los términos presentes en los fragmentos de texto de prueba.
- **Navegación en la interfaz:** La interfaz permite consultar el resumen de las ontologías, acceder a la jerarquía de clases, revisar propiedades y generar widgets de integración. Tras los ajustes en las referencias de las URLs de la API, estos elementos se muestran de forma consistente.

Funcionalidades Por Corregir

Durante la validación se detectaron algunas incidencias que, aunque solventadas de forma provisional, requieren soluciones más robustas:

- **Configuración de la URL de la API:** Se identificó la necesidad de separar la dirección interna (para llamadas del backend) y la externa (para peticiones desde el cliente).



Aunque esta medida permitió completar las consultas desde la interfaz, implica la exposición de la API mediante una URL pública. Este enfoque funciona en la fase actual, pero se recomienda evaluar alternativas que refuercen la seguridad y reduzcan dependencias de configuración manual.

- **Visualización de propiedades en la interfaz:** Inicialmente, las propiedades no se mostraban en la pestaña correspondiente debido a referencias codificadas hacia la URL de BioPortal. Tras modificar el frontend para utilizar variables de entorno configurables, el problema quedó resuelto. Se considera, no obstante, que esta solución debería consolidarse en fases posteriores para garantizar la mantenibilidad del código.
- **Servicio de anotación y búsqueda en la interfaz:** Se observó un problema similar al anterior: la dependencia de URLs fijas de BioPortal impedía mostrar los resultados de anotación y de la búsqueda. Una vez adaptado el código para utilizar la URL configurada en la API local, la funcionalidad quedó restituida.

Funcionalidades Para Implementar

Existen funcionalidades planificadas en el alcance del proyecto que aún no han sido desarrolladas en esta fase:

- Integración de servicios de inteligencia artificial proporcionados por el equipo de TeresIA, incluyendo extracción y enriquecimiento terminológico.
- Incorporación de búsquedas avanzadas mediante **Elasticsearch**, que permitan consultas más complejas y eficientes sobre el catálogo.
- Integración de un motor de grafos como **Neo4j**, con el objetivo de ofrecer una exploración basada en grafos de conocimiento.
- Migración del frontend actual hacia una solución basada en **React.js**, más ligera y escalable, que facilite el mantenimiento y la evolución del portal.
- Implementación de un sistema de gestión de usuarios con distintos roles y permisos, así como funcionalidades de validación y edición colaborativa de terminologías.

Plan para el Próximo Hito

El tercer hito se orientará a consolidar el catálogo de terminologías y a extender las capacidades ya verificadas en el presente entregable. En particular, las actividades previstas se centrarán en:

- Habilitar la **integración con servicios de búsqueda avanzada** mediante **Elasticsearch**, con el objetivo de permitir consultas más complejas, filtradas y eficientes sobre las ontologías disponibles en el portal.
- Incorporar **mecanismos de exploración de grafos** mediante la integración inicial con **Neo4j**, facilitando la navegación y el análisis de las relaciones semánticas entre términos.



- Evaluar la compatibilidad de estas integraciones con la arquitectura basada en microservicios definida en fases anteriores, garantizando la correcta comunicación entre componentes y la coherencia con los requisitos de interoperabilidad.
- Continuar con la **documentación detallada del proceso de despliegue**, de modo que las configuraciones y pruebas realizadas en esta fase puedan reproducirse de forma sistemática en entornos posteriores.

Este conjunto de tareas permitirá disponer de una versión ampliada del portal, con mayor capacidad de búsqueda y de análisis estructural de las ontologías, y servirá como base para la integración de los servicios de inteligencia artificial previstos en fases siguientes.

Conclusiones

La ejecución de este segundo hito ha permitido comprobar el funcionamiento integral de la infraestructura desplegada en la fase inicial. Se ha verificado que el portal puede gestionar ontologías en formato RDF/Turtle, procesarlas mediante el servicio NCBO Cron, indexarlas en Solr y ponerlas a disposición de los usuarios tanto a través de la API como desde la interfaz web.

El proceso seguido ha mostrado que los componentes principales de OntoPortal operan de manera estable tras la configuración y las modificaciones realizadas. Se han identificado incidencias relacionadas con la visualización de propiedades, la gestión de URLs codificadas y la dependencia de servicios externos para la representación de grafos. Estas limitaciones han sido mitigadas de forma provisional y se consideran aspectos a resolver en los próximos hitos.

Como resultado, se dispone ya de una primera versión operativa del catálogo de terminologías, junto con un inventario de funcionalidades confirmadas, elementos que requieren ajustes y desarrollos previstos. Este avance asegura una base tecnológica válida sobre la que continuar la integración de búsquedas avanzadas, exploración de grafos y servicios de inteligencia artificial en las siguientes fases del proyecto.

Anexo: Solución del uso de URLs estáticas de BioPortal

Durante la validación se detectó que en diferentes partes del código existían referencias fijas a la URL de producción de BioPortal (<http://data.bioontology.org>). Esta situación generaba problemas en el despliegue local y en entornos con dominios personalizados, ya que los enlaces generados no apuntaban a la API del propio portal TeresIA.

Inicialmente, se aplicó un parche que sustituía dinámicamente las referencias a la URL de BioPortal por la dirección configurada en el entorno local. Esta medida permitió continuar con las pruebas de ingestión e indexación, aunque se considera una solución temporal. Para resolver el problema de forma más estable se plantea realizar un **fork de las gemas de Ruby**



que gestionan los modelos y serializadores de OntoPortal. Con este enfoque se eliminarían las referencias codificadas de manera rígida, sustituyéndolas por **variables de entorno configurables**. Las modificaciones propuestas se centran en dos componentes principales: `Ontologies_linked_data` y `Ontologies_api_ruby_client`.

Ontologies_linked_Data¹

Esta gema define los modelos y serializadores de la API. En su configuración se empleaban las URIs de BioPortal como prefijos de los recursos RDF. En el fork se propone reemplazar dichas referencias por una URI configurable mediante variable de entorno, lo que permitirá utilizar tanto un despliegue local como un dominio específico en producción.

Ejemplo de verificación: accediendo al endpoint http://localhost:9393/ontologies/AS/latest_submission se comprobó que los enlaces generados ya apuntaban correctamente a la API local tras aplicar la modificación:

```
{
  + contact: [ ... ],
  - ontology: {
    - administeredBy: [
      http://api-agraph:9393/users/admin
    ],
    acronym: "AS",
    name: "aparells-sanitarios",
    @id: http://api-agraph:9393/ontologies/AS,
    @type: http://api-agraph:9393/metadata/Ontology,
    - links: {
      submissions: http://api-agraph:9393/ontologies/AS/submissions,
      properties: http://api-agraph:9393/ontologies/AS/properties,
      classes: http://api-agraph:9393/ontologies/AS/classes,
      single_class: http://api-agraph:9393/ontologies/AS/classes/{class_id},
      roots: http://api-agraph:9393/ontologies/AS/classes/roots,
      schemes: http://api-agraph:9393/ontologies/AS/schemes,
      collections: http://api-agraph:9393/ontologies/AS/collections,
      xl_labels: http://api-agraph:9393/ontologies/AS/skos_xl_labels,
      instances: http://api-agraph:9393/ontologies/AS/instances,
      metrics: http://api-agraph:9393/ontologies/AS/metrics,
      reviews: http://api-agraph:9393/ontologies/AS/reviews,
      notes: http://api-agraph:9393/ontologies/AS/notes,
      groups: http://api-agraph:9393/ontologies/AS/groups,
      categories: http://api-agraph:9393/ontologies/AS/categories,
      latest_submission: http://api-agraph:9393/ontologies/AS/latest_submission,
      projects: http://api-agraph:9393/ontologies/AS/projects,
      download: http://api-agraph:9393/ontologies/AS/download,
      views: http://api-agraph:9393/ontologies/AS/views,
      analytics: http://api-agraph:9393/ontologies/AS/analytics,
      ui: http://http://localhost:3000/ontologies/AS,
    - @context: {
      submissions: http://api-agraph:9393/metadata/OntologySubmission,
      properties: http://api-agraph:9393/metadata/Property,
      classes: http://www.w3.org/2002/07/owl#Class,
      single_class: http://www.w3.org/2002/07/owl#Class,
      roots: http://www.w3.org/2002/07/owl#Class,
      schemes: http://www.w3.org/2004/02/skos/core#ConceptScheme,
      collections: http://www.w3.org/2004/02/skos/core#Collection,
      xl_labels: http://www.w3.org/2008/05/skos-xl#Label,
      instances: http://api-agraph:9393/metadata/Instance,
      metrics: http://api-agraph:9393/metadata/Metrics,
      reviews: http://api-agraph:9393/metadata/Review,
      notes: http://api-agraph:9393/metadata/Note,
      groups: http://api-agraph:9393/metadata/Group,
      categories: http://api-agraph:9393/metadata/Category,
      latest_submission: http://api-agraph:9393/metadata/OntologySubmission,
      projects: http://api-agraph:9393/metadata/Project,
      download: http://api-agraph:9393/metadata/Ontology,
      views: http://api-agraph:9393/metadata/Ontology,
      analytics: http://api-agraph:9393/metadata/Analytics,
      ui: http://api-agraph:9393/metadata/Ontology
    }
  }
}
```

Figura 15: Respuesta en formato JSON a la petición GET a http://localhost:9393/ontologies/AS/latest_submission

También podemos consultar los metadatos directamente en nuestra API, por ejemplo navegando a <http://localhost:9393/metadata/Ontology>:

¹ https://github.com/ontoportal/ontologies_linked_data



<http://api-agraph:9393/metadata/Ontology>

Resource collection: [/ontologies](#)

HTTP Methods for Resource

HTTP Verb	Path
GET	/ontologies
GET	/ontologies/acronym
GET	/ontologies/acronym/latest_submission
GET	/ontologies/acronym/download
GET	/ontologies_full
PUT	/ontologies/acronym
POST	/ontologies/acronym/pull
POST	/ontologies
PATCH	/ontologies/acronym
DELETE	/ontologies/acronym

Figura 16: Listado de metadatos, en formato tabla, de una ontología.

Ontologies_api_ruby_client²

Esta librería define los modelos y serializadores que utiliza el frontend para interactuar con la API. En su versión original, el parámetro *media_type* incluía la URL fija de BioPortal. Se adaptó el código para que este valor se establezca dinámicamente a partir de la variable *rest_url* definida en la configuración del frontend.

Asimismo, fue necesario ajustar el tratamiento de los enlaces en otras partes del código, ya que dependían de la estructura previa de las URIs. Tras estas modificaciones, la aplicación mostró de manera consistente los enlaces a la API desplegada en TeresIA.

Con esta estrategia se evita la dependencia de servicios externos y se asegura la flexibilidad necesaria para operar en diferentes entornos, manteniendo la coherencia de los recursos expuestos en el portal.

² https://github.com/ontoportal/ontologies_api_ruby_client